Session 5: RCT, Data Joint and Geospatial Data

Control and Treatment, Data joint, Geospatial Data

Karry Jiao
dj398@cam.ac.uk
KJ Github

Nov. 2026

Table of contents

1. Introduction
2. Lecture review
3. Randomization
4. Data Joint
5. Geospatial Mapping
6. Key Takeaways

Introduction

What this session is about

By the end, you should be able to:

- Do random selection with R;
- Master different functions for RCT;
- Know how to joint datasets
- Have a general idea of geospatial data in R

Examples use Census data, and randomly generated sample dataframe.

Open your existing RStudio project and create `exercises-session5.R`.

Lecture Review

Lecture Review — Why randomize?

An observed difference between treated and untreated groups may reflect both a **causal effect** and **selection bias**.

Observed difference = ATT + Selection bias

- **Random assignment** makes treatment independent of potential outcomes, removing systematic selection bias.
- Treatment and control groups are comparable **in expectation**, including on unobserved characteristics.

Randomization does not make everyone identical or guarantee perfectly balanced groups in a particular sample.

- **World Bank Project: Nigeria's YouWiN!:** Randomly allocate grants to a subset of eligible entrepreneurs to estimate effects on business growth and employment.
- **World Bank Project: Business training in West Africa:** Randomly assign entrepreneurs to personal initiative training, traditional training, or a control group to compare business profits.
- **CDRF's China REACH:** Randomly select participants from the "one village one kindergarten project" to the treatment group and those kids from different villages without attending kindergartens to the control group.

Lecture Review — Treatment and control

Lecture example: Do new parks improve residents' mental health?

Experimental design

- Start with **200 eligible neighbourhoods**.
- Randomly assign **100** to receive a new park.
- The remaining **100** form the control group.
- Measure mental health in both groups after the intervention.

Why does this help?

The control group provides an estimate of what would have happened to the treatment group **without the intervention**.

Check baseline characteristics to describe balance.

Randomization can produce chance differences. Insignificant balance tests do not prove that assignment was random.

Lecture Review — Estimating the effect

With full compliance and no spillovers, estimate the average treatment effect using:

$$\widehat{\mathrm{ATE}} = \bar{Y}_T - \bar{Y}_C$$

The equivalent regression is:

$$Y_i = \alpha + \beta T_i + \varepsilon_i$$

- $T_i = 1$ for treatment and $T_i = 0$ for control.
- α is the control group's mean outcome.
- β is the difference in mean outcomes between the two groups.

Pre-treatment controls can improve precision. Random assignment provides the basis for causal identification.

Lecture Review — Practical challenges

Challenge	Implication for the study
Non-compliance	Estimate the **intention-to-treat (ITT)** effect by comparing groups as originally assigned.
Missing follow-up outcomes	Differential attrition can undermine comparability. ITT alone does not solve missing data.
Spillovers	Treatment may affect the control group. Consider randomizing clusters such as schools or villages.
External validity	Effects may differ across populations and settings. Consider mechanisms and context.
Ethics and statistical power	Justify assignment, protect participants, and plan a sufficient sample size.

Random assignment supports causal inference, but implementation and follow-up still matter.

Randomization

Random selection with sample()

`sample()` randomly selects elements from a vector.

```
# Randomly select 4 numbers from 1 to 10
sample(x = 1:10, size = 4, replace = FALSE)
```

```
## [1]  2  8  3 10
```

```
# Randomly select 3 letters
sample(x = letters, size = 3, replace = FALSE)
```

```
## [1] "u" "o" "g"
```

- `x`: the values to select from.
- `size`: how many values to select.
- `replace = FALSE`: each element can be selected at most once.

If you omit size, sample() shuffles all elements of x.

Random Sampling

Generate a dataset with random sampling

```
set.seed(14)

df1 ← data.frame (sample(1:20, 30, replace=TRUE))

df2← data.frame (replicate(5, sample(1:20, 30, replace=TRUE)))
```

- `14` is an arbitrary choice. Other seed numbers also work.
- Resetting the same seed reproduces the same sequence of random draws.
- Without resetting, subsequent random calls continue along the sequence.

Random Sampling with the Census Data

Each row of `census` represents one state.

```
# Basic-R checking
c(n_rows = nrow(census),
  n_states = length(unique(census$state)))

# Display selected variables for the first 5 states
head(
  census[c("state", "region", "pop", "medage")],
  5
)
```

Now suppose the federal government wants to test a new program:

- **Treatment:** states offered the programme.
- **Control:** states not offered the programme during the trial.

Random Sampling

First, randomly select **10 states** from our dataset.

```
set.seed(14)

# Draw 10 row numbers without replacement
selected_rows ← sample(
  x = seq_len(nrow(census)), #
  size = 10,
  replace = FALSE
)

# Keep those rows and selected columns
census_sample ← census[
  selected_rows,
  c("state", "region", "pop", "medage")
]

census_sample
```

```
##          state  region      pop medage
## 9      Florida   South  9746324   34.7
## 50     Wyoming    West   469557   27.1
## 11      Hawaii    West   964691   28.4
## 43       Texas   South 14229191   28.2
## 14     Indiana N Cntrl  5490224   29.2
## 16      Kansas N Cntrl  2363679   30.1
## 26     Montana    West   786690   29.0
## 33 N. Carolina   South  5881766   29.6
## 41   S. Dakota N Cntrl   690768   28.9
## 3       Arizona    West  2718215   29.2
```

Random Assignment

Give each state a **50% chance** of assignment to treatment.

```
set.seed(14)

census_sample$treat ← sample(
  x = c(0, 1),
  size = nrow(census_sample),
  replace = TRUE,
  prob = c(0.5, 0.5)
)

table(census_sample$treat)
```

```
set.seed(14)
census_sample ←census_sample %>%
  mutate(
    treat= sample(
    x = c(0, 1),
    size = n(),
    replace = TRUE,
    prob = c(0.5, 0.5)
  )
  )
```

- `0` means control; `1` means treatment.
- Every state has the same treatment probability.
- The total number of treated states can vary across assignments.

Random assignment

Another way to assign treatment and control group with fixed group size, is by creating a dummy variable. Shuffle a vector containing **25 zeros and 25 ones**.

```
# in basic R:
census_rct ← census[order(census$state), ]
# with dplyr:
census_rct ← census %>%
  arrange(state)
```

```
set.seed(14)

census_rct$treat ← sample(
  x = rep(c(0, 1), each = 25), # create a vector with 25 "0" and 25 "1"
  replace = FALSE
)
```

Then you can check who is in treatment and control:

```
census_rct$state[census_rct$treat == 1]

census_rct$state[census_rct$treat == 0]
```

Random assignment

And if you want add readable group labels

```
census_rct$assign_label ← factor(
  census_rct$treat,
  levels = c(0, 1),
  labels = c("Control", "Treatment")
)

# or:
census_rct ← census_rct %>%
  mutate(
    group = factor(
      treat,
      levels = c(0, 1),
      labels = c("Control", "Treatment")
    )
  )
```

Packages for randomization and analysis

There are some R packages designed for doing randomization

Package	Purpose	Example functions
randomizr	Assign treatment under different designs	`complete_ra()`, `block_ra()`
estimatr	Estimate effects with robust standard errors	`lm_robust()`, `difference_in_means()`

```
# Run once in the Console if needed:
# install.packages(c("randomizr", "estimatr"))

library(randomizr)
library(estimatr)
```

randomizr: complete random assignment

Use `complete_ra()` to assign exactly 25 states to treatment.

```
census_rct2 ← census_rct%>% select(-treat)

set.seed(14)

census_rct2$treat ← complete_ra(
  N = nrow(census_rct2),
  m = 25
)
```

- `N`: total number of units.
- `m`: number assigned to treatment.
- The function returns a vector containing `0` and `1`.

This implements the same type of design as shuffling 25 zeros and 25 ones. Yet different algorithms can produce different assignments even with the same seed.

Estimate the treatment effect

Exercise

- Estimate the difference of a variable between the treatment and control group.
- Hint: First, calculate the difference. Then do a quick linear regression check.

Estimate the treatment effect

Exercise

- Estimate the difference of a variable between the treatment and control group.
- Hint: First, calculate the difference. Then do a quick linear regression check.

Solution:

```
census_rct ← census_rct %>%
  mutate(marg_rate = marriage / pop)

# Treatment mean minus control mean
census_rct %>% summarise(
    difference = mean(marg_rate[treat == 1]) -
      mean(marg_rate[treat == 0]))
```

```
fit_ols ← lm( marg_rate ~ treat, data = census_rct)
coef(fit_ols)
```

The `treat` coefficient equals the difference in sample means.

Estimatr: Robust SE

Estimate the effect and quantify its uncertainty.

```
fit_robust ← lm_robust(
  marg_rate ~ treat,
  data = census_rct,
  se_type = "HC2"
)

# Display the treatment estimate and confidence interval
summary(fit_robust)
```

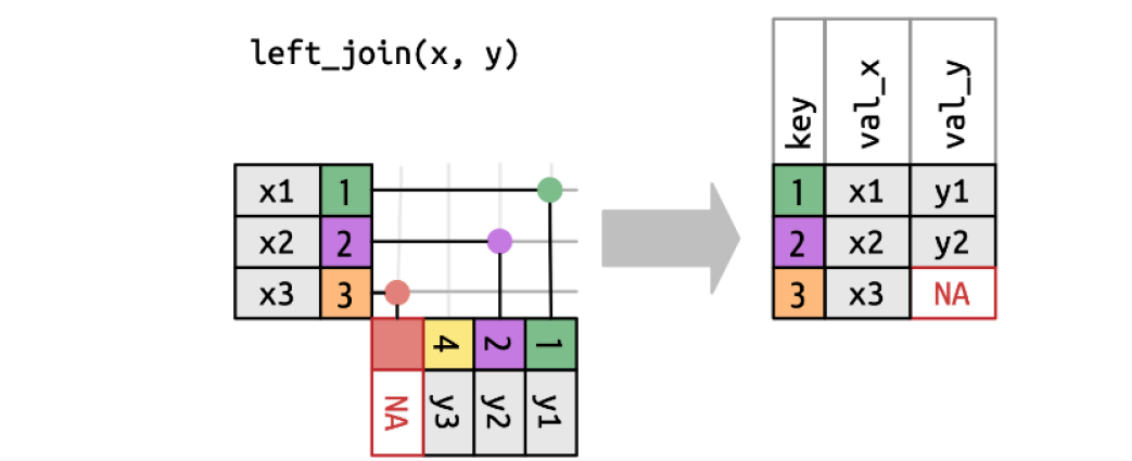
- **Estimate:** the treatment–control difference in mean outcomes.
- **Coefficient of treat:** states assigned to the treatment group have an average marriage rate 0.57% lower than that of the states in the control group.

Data Joint

Data Joint

We can use the `dplyr::left_join()` function to merge two dataframes. The function syntax is: `left_join(a_df, another_df, by = c("id_col1"))`.

> A left join takes all the values from the first table, and looks for matches in the second table. If it finds a match, it adds the data from the second table; if not, it adds missing values. It is the equivalent of `merge, keep(master matched)` in Stata.

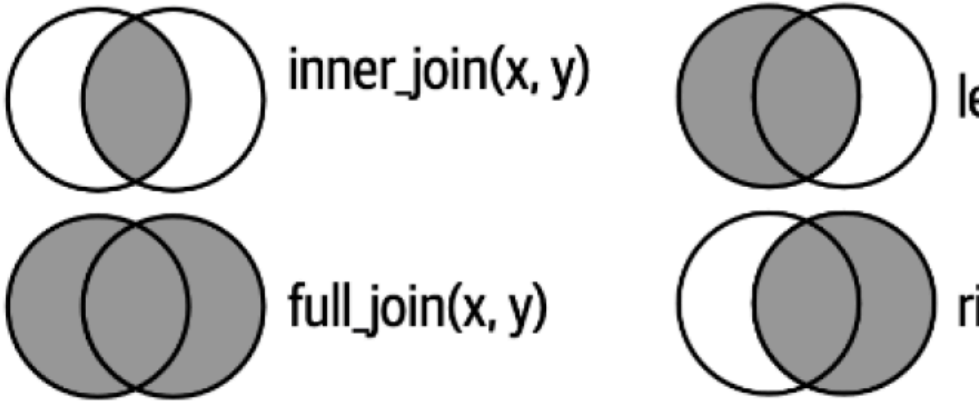


Left, Right, Inner and Full Joins

```
a ← tibble(id = c(1, 2, 3, 4), score = sample(1:10, 4, replace=FALSE))
b ← tibble(id = c(2, 3, 5), region = c("East", "West","Nowhere"))
```

Command	IDs retained	What happens to unmatched observations?
`left_join(a, b, by = "id")`	1, 2	Missing region for ID 1
`right_join(a, b, by = "id")`	2, 3	Missing score for ID 3
`inner_join(a, b, by = "id")`	2	Unmatched observations excluded
`full_join(a, b, by = "id")`	1, 2, 3	Missing values where a match is absent

Left, Right, Inner and Full Joins



Merging Data Sets with Basic R Function: merge()

In Basic R, you can also merge dataset with the function merge()

```
merge(x, y, by = "key", all = FALSE)  --- Inner joint
merge(x, y, by = "key", all = TRUE)  --- Full joint
merge(x, y, by = "key", all.x=TRUE)  --- Left joint
merge(x, y, by = "key", all.y = TRUE)  --- Right joint
```

For example:

```
merge(a,b, by="id", all=TRUE)
```

Merging Data Sets

Exercise: Join the dataframes: `regions` and `whr17`.

Merging Data Sets

Exercise: Join the dataframes: `regions` and `whr17`.

Solution:

```
whr17 ← whr17 %>%
  left_join(regions, by = "country") %>%
  select(country, region, everything())
```

Notes:

Look at the `everything()` function. It takes all the variables from the dataframe and put them after country and region. In this way, select can be use to **order** columns!

Intro to Geospatial Mapping

Intro to Geospatial Mapping

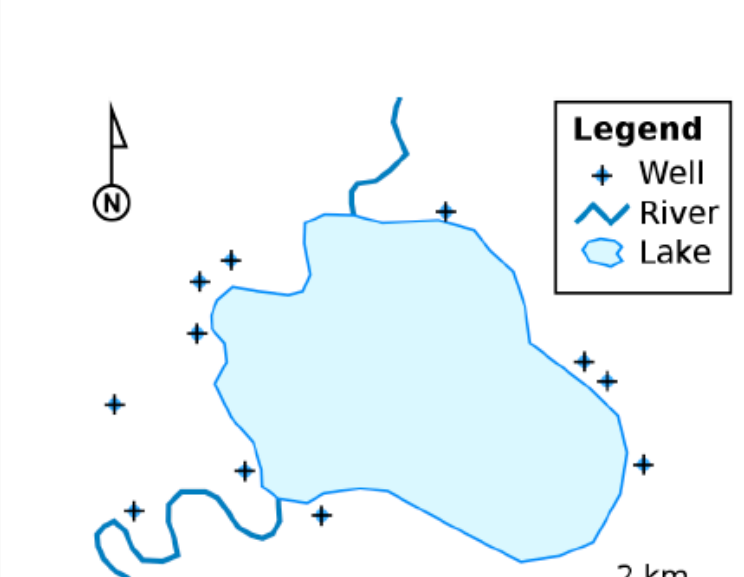
- This session could be a whole course on its own, but we only have an hour and half.
- To narrow our subject, we will focus on only one type of spatial data, vector data.
- This is the most common type of spatial data that non-GIS experts will encounter in their work.
- We will use the `sf` package, which is the tidyverse-compatible package for geospatial data in R.

Overview of GIS conceps

Spatial data: The two main types of spatial data are **vector data** and **raster data**

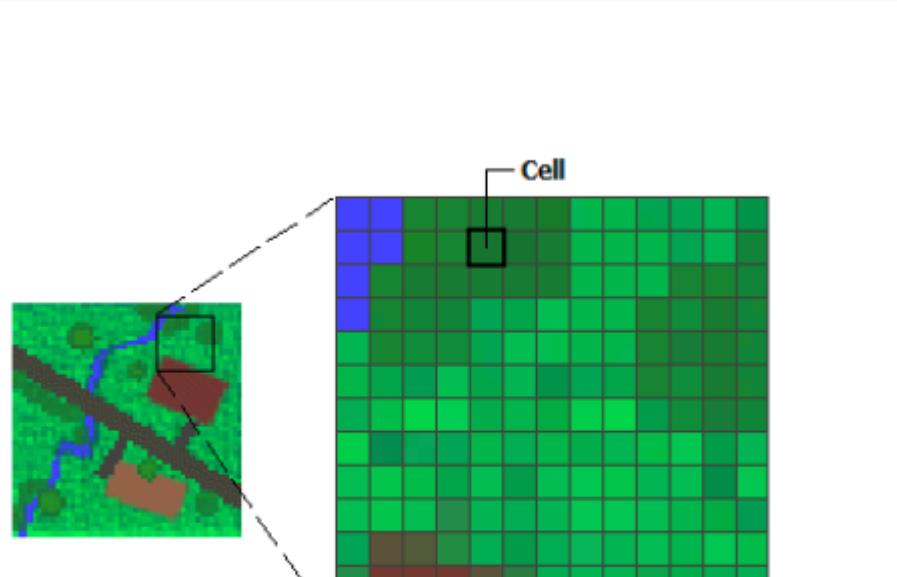
Vector data

- Points, lines, or polygons
- Common file formats include shapefiles (.shp) and geojsons (.geojson)
- Examples: polygons on countries, polylines of roads, points of schools



Raster data

- Spatially referenced grid
- Common file format is a geotif (.tif)
- Example: Satellite imagery of nighttime lights



Setup

Install new packages

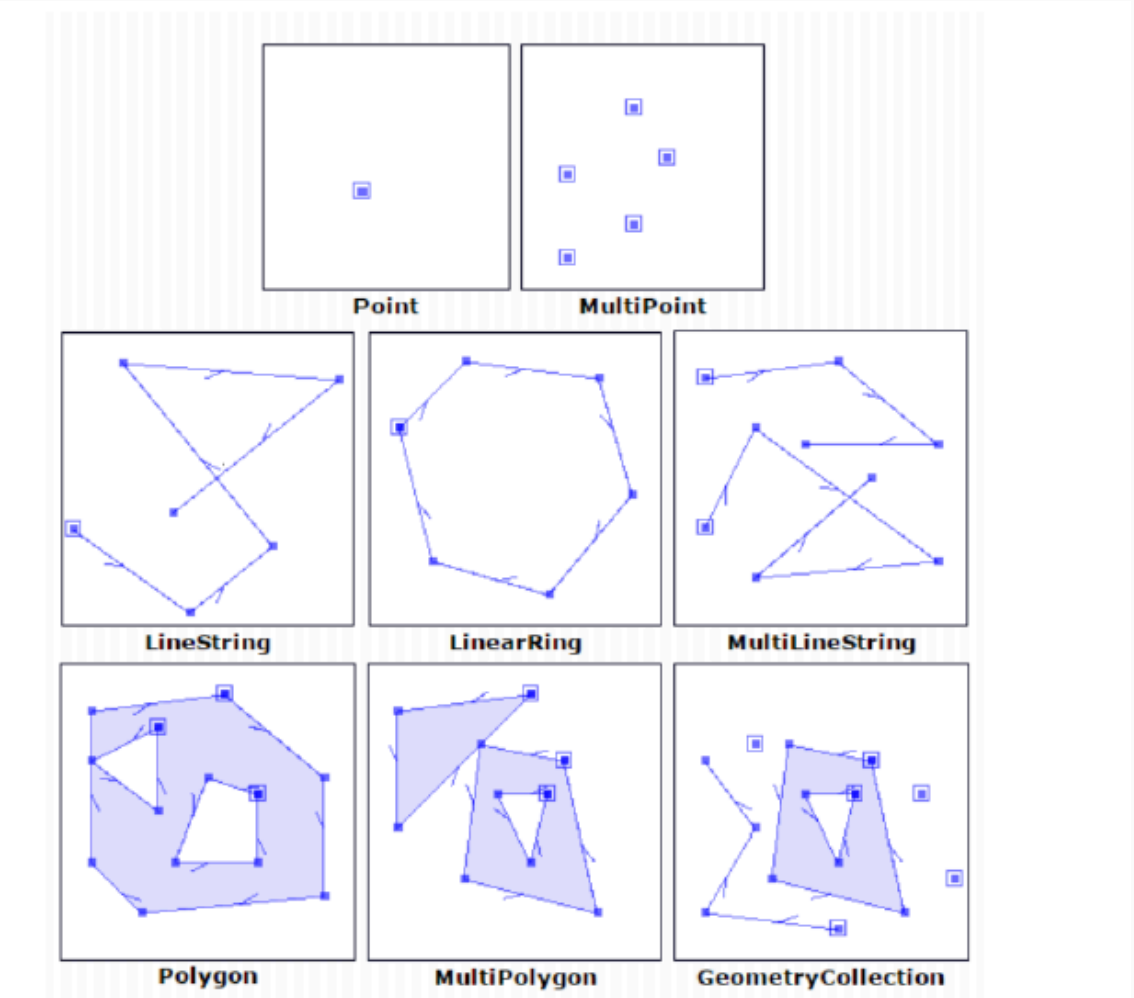
```
install.packages(c("sf",
                   "leaflet",
                   "geosphere"),
                 dependencies = TRUE)
```

And load them

```
library(here)
library(tidyverse)
library(sf)        # Simple features
library(leaflet)   # Interactive map
library(geosphere) # Great circle distances
```

Load and explore polylines, polylines, and points

The main package we'll rely on is the `sf` (simple features) package. With `sf`, spatial data is structured similarly to a **dataframe**; however, each row is associated with a **geometry**. Geometries can be one of the below types.



Load and explore polygon

The first thing we will do in this session is to recreate this data set:

```
country_sf ←
  st_read(here("DataWork",
               "DataSets",
               "Final",
               "country.geojson"))
```

```
## Reading layer `country' from data source
##   `/Users/karryjiao/Desktop/R training/Bennett R Training/bennettschool_statistics_TA/code/MPP-r-training-2026/DataWork
##   using driver `GeoJSON'
## Simple feature collection with 300 features and 13 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: 33.90959 ymin: -4.720417 xmax: 41.92622 ymax: 5.061166
## Geodetic CRS:  WGS 84
```

Exploring the data

Look at first few observations

```
head(country_sf)
```

```
## Simple feature collection with 6 features and 13 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: 35.52292 ymin: -0.198901 xmax: 36.29659 ymax: 0.990413
## Geodetic CRS:  WGS 84
##       GID_2 GID_0 COUNTRY   GID_1  NAME_1 NL_NAME_1          NAME_2 VARNAME_2
## 1 KEN.1.1_1   KEN   Kenya KEN.1_1 Baringo      <NA>             805      <NA>
## 2 KEN.1.2_1   KEN   Kenya KEN.1_1 Baringo      <NA> Baringo Central      <NA>
## 3 KEN.1.3_1   KEN   Kenya KEN.1_1 Baringo      <NA>   Baringo North      <NA>
## 4 KEN.1.4_1   KEN   Kenya KEN.1_1 Baringo      <NA>   Baringo South      <NA>
## 5 KEN.1.5_1   KEN   Kenya KEN.1_1 Baringo      <NA>   Eldama Ravine      <NA>
## 6 KEN.1.6_1   KEN   Kenya KEN.1_1 Baringo      <NA>         Mogotio      <NA>
##   NL_NAME_2       TYPE_2    ENGTYPE_2 CC_2 HASC_2
## 1      <NA> Constituency Constituency  162   <NA>
## 2      <NA> Constituency Constituency  159   <NA>
## 3      <NA> Constituency Constituency  158   <NA>
## 4      <NA> Constituency Constituency  160   <NA>
## 5      <NA> Constituency Constituency  162   <NA>
## 6      <NA> Constituency Constituency  161   <NA>
##                       geometry
```

Exploring the data

Number of rows

```
nrow(country_sf)
```

```
## [1] 300
```

Exploring the data

Check coordinate reference system

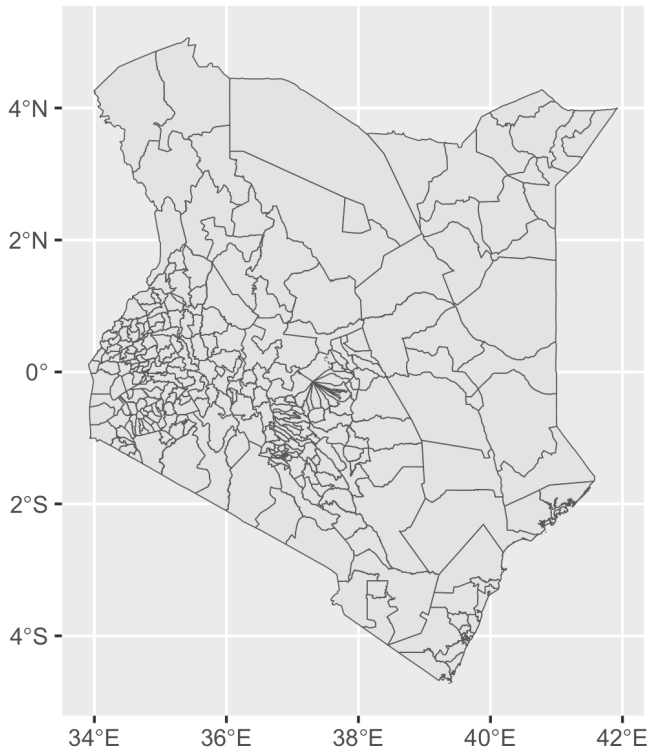
```
st_crs(country_sf)
```

```
## Coordinate Reference System:
##   User input: WGS 84
##   wkt:
## GEOGCRS["WGS 84",
##     DATUM["World Geodetic System 1984",
##         ELLIPSOID["WGS 84",6378137,298.257223563,
##             LENGTHUNIT["metre",1]]],
##     PRIMEM["Greenwich",0,
##         ANGLEUNIT["degree",0.0174532925199433]],
##     CS[ellipsoidal,2],
##         AXIS["geodetic latitude (Lat)",north,
##             ORDER[1],
##             ANGLEUNIT["degree",0.0174532925199433]],
##         AXIS["geodetic longitude (Lon)",east,
##             ORDER[2],
##             ANGLEUNIT["degree",0.0174532925199433]],
##     ID["EPSG",4326]]
```

Exploring the data

Plot the data. To plot using `ggplot2`, we use the `geom_sf` geometry.

```
ggplot() +
  geom_sf(data = country_sf)
```



Attributes of data

We want the area of each location, but we don't have a variable for area

```
names(country_sf)
```

```
##  [1] "GID_2"     "GID_0"     "COUNTRY"   "GID_1"     "NAME_1"    "NL_NAME_1"
##  [7] "NAME_2"    "VARNAME_2" "NL_NAME_2" "TYPE_2"    "ENGTYPE_2" "CC_2"
## [13] "HASC_2"    "geometry"
```

Attributes of data

Determine area. Note the CRS is spherical (WGS84), but `st_area` gives area in meters squared. R uses s2 geomety for this.

```
st_area(country_sf)
```

```
## Units: [m^2]
##   [1]   174612548   664171032  1640235288  1893541200   909726196  1162391322
##   [7]  4446442673   244343146   319588761   546475821   791703635   487034695
##  [13]   347077424   234870080   320531683   178295354   302805234   267396640
##  [19]   950722974   208724111   377069605   235040170   166654802   311166355
##  [25]   244842707   270856300   195816268   234613620   250070745   300226047
##  [31]   544675636   890441628   819095221   562423505   473818437   774008353
##  [37]  1329273270   340468166  4248579388  7557295525 16712310204   603028994
##  [43]  8442761951  6483073596   270237144   247974418   626536513   259329880
##  [49]  1032562720   699140857   292011673  1147576864   124531035 15430646699
##  [55] 10122778443  4333824029  3283326674   112849413  6385975697  7975956018
##  [61]   210204075   144513355   145118575   102257259   162397218   424754069
##  [67]   278291541   175812288   138974871   258559234   412385494   246642948
##  [73]   447240937   317315791   755656427   354432319   466560197   244100322
##  [79]   214221442   184422278   336670907    59209638    79588059   106276706
##  [85]   172477093   468135485   283853034   204126728   232219564  2950195157
##  [91]   703903982   832269572   436237078  6970056180   614090663   201415848
##  [97]   544469250   238375984   550503547   230810950   241653749   114996036
## [103]   102453209   127269592   131050389    98022999   136597611   160116397
## [109]   208693101    72147288   162435042   313730881   667098008   404328975
```

Operations similar to dataframes

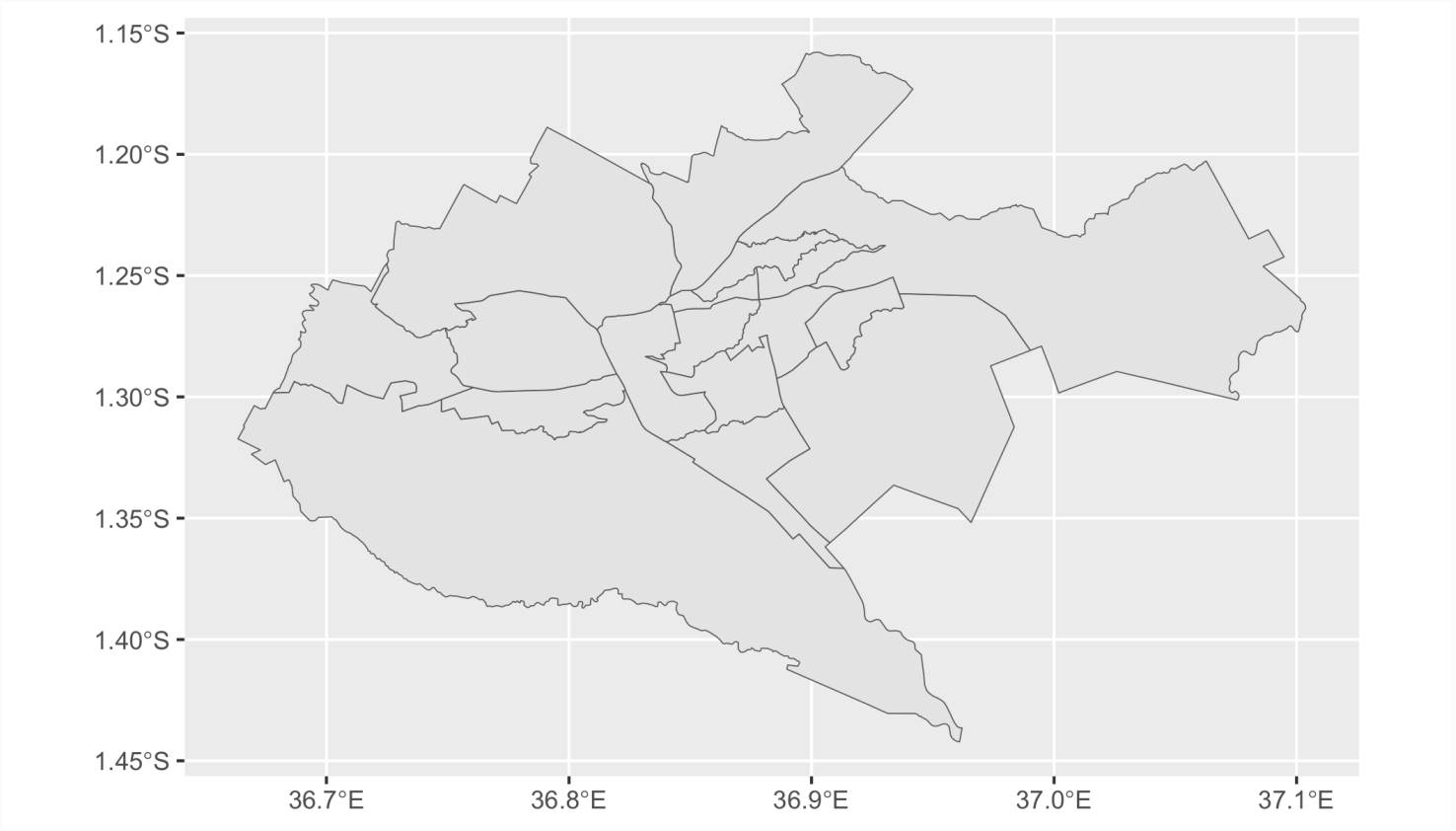
Create new dataset that captures locations for one administrative region

```
city_sf ← country_sf %>%
  filter(NAME_1 == "Nairobi")
```

Operations similar to dataframes

Plot the dataframe

```
ggplot() +
  geom_sf(data = city_sf)
```



Load and explore polyline

Exercise:

- Load the roads data `roads.geojson` and name the object `roads_sf`
- Look at the first few observations
- Check the coordinate reference system
- Map the polyline

Load and explore polyline

Exercise:

- Load the roads data `roads.geojson` and name the object `roads_sf`
- Look at the first few observations
- Check the coordinate reference system
- Map the polyline

Solution:

```
roads_sf ← st_read(here("DataWork", "DataSets", "Final", "roads.geojson"))

head(roads_sf)

st_crs(roads_sf)

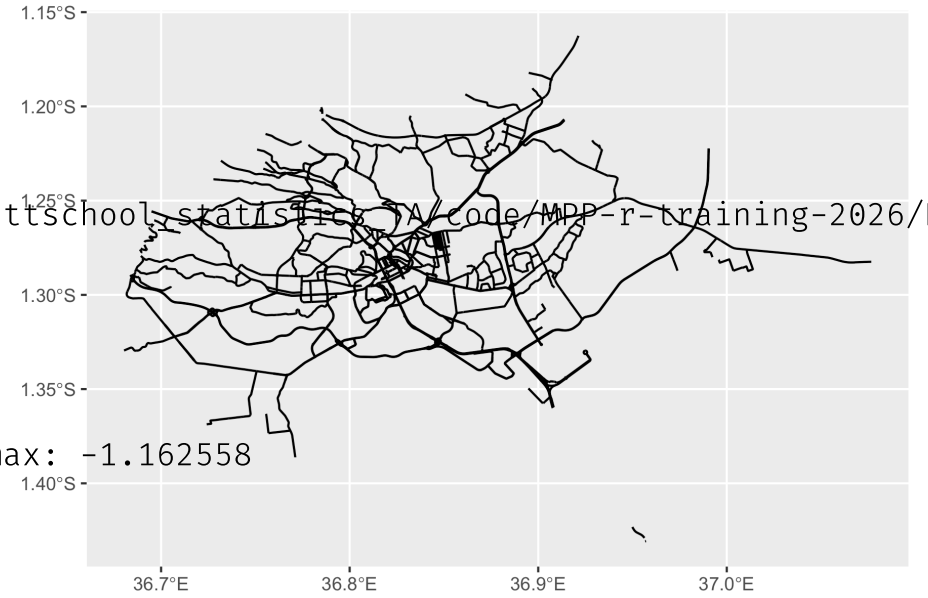
ggplot() +
  geom_sf(data = roads_sf)
```

Load and explore polyline

```
roads_sf ←
  st_read(here("DataWork",
               "DataSets",
               "Final",
               "roads.geojson"))
```

```
## Reading layer `roads' from data source
##   `/Users/karryjiao/Desktop/R training/Bennett R Training/bennettschool-statistics-TA/code/MPP-r-training-2026/DataWork
##   using driver `GeoJSON'
## Simple feature collection with 3326 features and 3 fields
## Geometry type: MULTILINESTRING
## Dimension:     XY
## Bounding box:  xmin: 36.68034 ymin: -1.430759 xmax: 37.07664 ymax: -1.162558
## Geodetic CRS:  WGS 84
```

```
ggplot() +
  geom_sf(data = roads_sf)
```



Load and explore polyline

Exercise: Determine length of each line (hint: use `st_length`)

Load and explore polyline

Exercise: Determine length of each line (hint: use `st_length`)

Solution:

```
st_length(roads_sf)
```

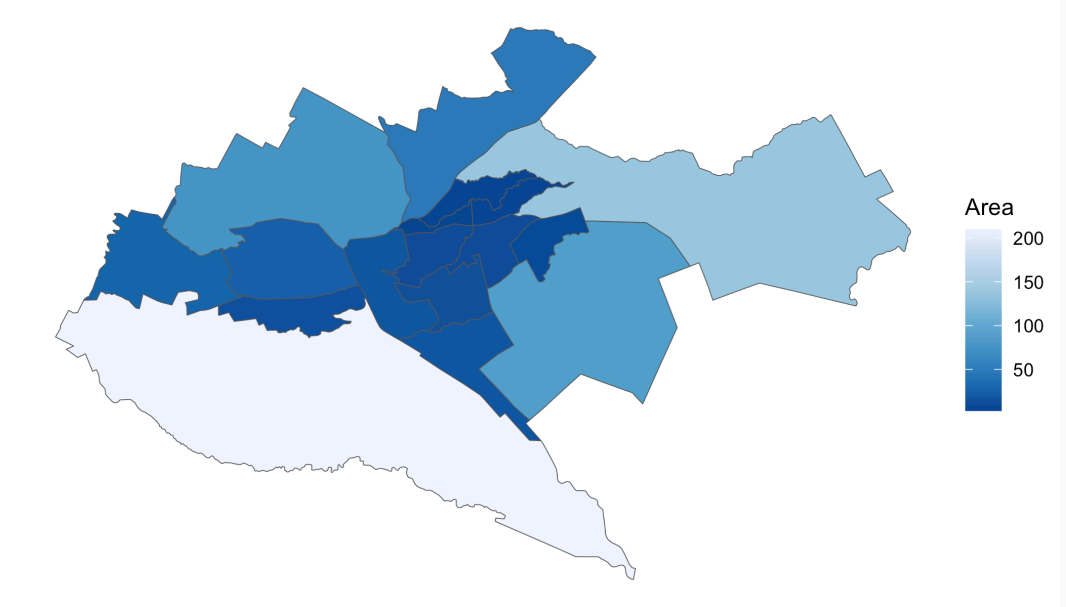
```
## Units: [m]
##    [1]  901.5756687  137.8591461  166.0896978   24.2633581  174.3557406
##    [6]  482.5503262  486.2506610   64.1042587  615.4574212   16.7135334
##   [11]   19.6987155    3.8487318    4.1237787  555.4454885  551.5809255
##   [16]    7.0648280  229.5654887  588.9304516  136.9445835  579.1322038
##   [21]   58.3317564   21.7936436   90.1570913   41.2507165   81.7085529
##   [26]   68.3241690  496.3577265   14.1516418   44.0357348   45.8100172
##   [31]   41.6768610   35.1260485   40.5775920   43.1833377 1068.7247201
##   [36]  254.9862449  554.1882417  280.0103135  573.6525111  632.2063080
##   [41]  903.4320724   15.2156667   84.9916400   89.5853644   39.3628302
##   [46]  656.4399014  562.9941828   53.0465233   55.0490837   22.1513814
##   [51]  157.2766915  742.1361789  255.6841254  188.6658952  196.8677316
##   [56]  322.5741540  136.8794494  145.1362286 1123.8795488  449.7621126
##   [61]   76.8128399  425.6253062   83.2814112   67.4804517   68.9621063
##   [66]  537.9023283   19.2785758  103.2281926   59.2140086   26.1598809
```

Make better static map

Lets make a better static map.

```
# Adding a variable with squared km
city_sf ← city_sf %>%
  mutate(area_m = city_sf %>% st_area() %>% as.numeric(
         area_km = area_m / 1000^2)

# Plotting
ggplot() +
  geom_sf(data = city_sf,
          aes(fill = area_km)) +
  labs(fill = "Area") +
  scale_fill_distiller(palette = "Blues") +
  theme_void()
```



Another static map

Exercise: Make a static map of roads, coloring each road by its type. (**Hint:** The `highway` variable indicates the type).

Another static map

```
ggplot() +
  geom_sf(data = roads_sf,
          aes(color = highway)) +
  theme_void() +
  labs(color = "Road Type")
```



Key Takeaways

Key Takeaways

- This session goes over the random sampling and assignment in R, sample(), etc.
- Also we recall how to merge two datasets using left/right/inner/full joint and merge()
- A brief introduction on the geospatial data and how to create the static graphs using geom_sf along with ggplot()